

Московский Авиационный Институт  
(Национальный Исследовательский Университет)  
Институт №8 “Компьютерные науки и прикладная математика”  
Кафедра №806 “Вычислительная математика и программирование”

**Лабораторная работа №3 по курсу**  
**«Операционные системы»**

Группа: М8О-211Б-23

Студент: Сергеева А. А.

Преподаватель: Бахарев В.Д. (ФИИТ)

Оценка: \_\_\_\_\_

Дата: 09.10.24

Москва, 2024

# Постановка задачи

## Вариант 1.

Пользователь вводит команды вида: «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип `int`. Количество чисел может быть произвольным.

## Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; – создает дочерний процесс.
- `int execv (const char * path, char *const argv[])` – вызов для замещения тела процесса, в случае успешного выполнения системного вызова виртуальное адресное, пространство процесса полностью заменяется. В `path` записывается абсолютный или относительный путь к исполняемой файлу для запуска, в `argv` массив аргументов командной строки.
- `ssize_t write (int fd, const void * buf, size_t n)` – записывает `n` бит из указанного буфера в файл, соответствующий файловому дескриптору, возвращает количество записанных байт в случае успеха, иначе `-1`.
- `ssize_t read (int fd, void * buf, size_t nbytes)` – считывает `nbytes` из файла, соответствующего файловому дескриптору `fd` в буфер `buf`, возвращает количество прочитанных байт, `0` – если достигнут конец файла, `-1` в случае ошибки.
- `int open(const char *pathname, int oflag, ... /* mode_t mode */)` – Открывает файл в соответствие с указанными модами, возвращает дескриптор файла в случае успеха, `-1` в случае ошибки.
- `int close (int fd)` – закрывает файл.
- `pid_t wait(&pstatus)` – приостанавливает выполнение текущего процесса до тех пор, пока какой-либо сыновний процесс не завершит своё выполнение, либо пока в текущий процесс не поступит сигнал, который вызовет обработчик сигнала или завершит выполнение процесса, записывает статус завершенного процесса в `pstatus`
- `int shm_open(const char *name, int flags, mode_t mode)` – создает и открывает новый (или открывает уже существующий) объект разделяемой памяти POSIX. создание или открытие совместно используемой памяти, `name` - имя объекта, `flags` - определяет задаваемый объект, `mode` - задает режим доступа к объекту. При нормальном завершении работы `shm_open` возвращает неотрицательный описатель файла. При ошибках `shm_open` возвращает `-1`.
- `void * mmap(void *start, size_t length, int prot, int flags, int fd, off_t offset)` – отражает `length` байтов, начиная со смещения `offset` файла (или другого объекта), определенного файловым описателем `fd`, в память, начиная с адреса `start`. Последний параметр (адрес) необязателен, и обычно бывает равен `0`. Настоящее местоположение отраженных данных возвращается самой функцией `mmap`. Аргумент `prot` описывает желаемый режим защиты памяти. Параметр `flags` задает тип отражаемого объекта,

опции отражения и указывает, принадлежат ли отраженные данные только этому процессу или их могут читать другие.

- `int ftruncate(int fd, off_t length)` – устанавливает длину файла с дескриптором `fd` в `length` байт.
- `int shm_unlink(const char *name)` – удаляет объект, предварительно созданный с помощью `shm_open`.
- `int munmap(void *start, size_t length)` – освобождает ранее выделенные страницы памяти, начиная с адреса `start` и размером `length` байт, с округлением вверх до размера страницы памяти.
- `sem_init(sem_t *sem, int pshared, unsigned int value)` – инициализирует объект семафора `sem`, `pshared` - указывает, будет ли семафор общим для потоков или процессов. Если равен 0, то семафор - общий для потоков, если ненулевое значение - то семафор общий для процессов, `value` - начальное значение семафора.
- `int sem_post(sem_t *sem)` – увеличивает (разблокирует) семафор. Если значение семафора после этого становится больше нуля, то другой поток, который ожидает получение семафора и заблокирован с помощью вызова `sem_wait()`, проснется и заблокирует семафор.
- `int sem_wait(sem_t *sem)`: уменьшает (блокирует) семафор. Если значение семафора больше нуля, то выполняется уменьшение, и поток продолжает работу. Если значение семафора равно нулю, то поток блокируется до тех пор, пока не станет возможным выполнить уменьшение.
- `int sem_destroy(sem_t *sem)`: удаляет семафор

Первым аргументом командной строки пользователь пишет имя файла, в котором будет записан ответ. В `server.c` создаем или открываем объект совместно используемой памяти. Устанавливаем размер совместно используемой памяти с помощью `ftruncate`. Отображаем совместно используемую память в адресное пространство родительского процесса. Создаем дочерний процесс с помощью `fork`. Родительский процесс считывает ввод с консоли, сохраняет ввод в общую память и сигнализирует дочернему процессу.

Дочерний процесс маппирует память и ожидает сигнала от родителя. Выполняет сложение чисел, выводит их в результирующий файл. Очищаем использованные ресурсы.

## Код программы

### posix\_ipc-client.c

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <fcntl.h>
```

```
#include <sys/mman.h>

#include <semaphore.h>

#include <unistd.h>

#include <sys/stat.h>

#include <ctype.h>

#include <stdint.h>


#define SHM_NAME "/my_shared_memory"

#define SEM_NAME "/my_semaphore"


int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        char msg[1024];

        uint32_t len = snprintf(msg, sizeof(msg), "usage: %s filename, wrong
cnt_arg\n", argv[0]);

        write(STDERR_FILENO, msg, len);

        exit(EXIT_FAILURE);
    }

    char *filename = argv[1];

    sem_t *semaphore = sem_open(SEM_NAME, 0);

    if (semaphore == SEM_FAILED)
    {
        perror("error: failed to open semaphore");

        exit(EXIT_FAILURE);
    }

    int shm_fd = shm_open(SHM_NAME, O_RDONLY, 0);
```

```

if (shm_fd < 0)
{
    perror("error: failed to open shared memory");
    sem_close(semaphore);
    exit(EXIT_FAILURE);
}

char *shm_ptr = mmap(0, 1024, PROT_READ, MAP_SHARED, shm_fd, 0);
if (shm_ptr == MAP_FAILED)
{
    perror("error: failed to map shared memory");
    shm_unlink(SHM_NAME);
    sem_close(semaphore);
    exit(EXIT_FAILURE);
}

if (sem_wait(semaphore) == -1)
{
    perror("error: failed sem_wait");
}

char *input = shm_ptr;

int j = 0, length = 0;
long int res = 0, sum = 0;
char msg[33];
char num[20];
for (int i = 0; i < strlen(input); ++i)
{
    if (!isspace(input[i]))
    {

```

```

        num[j++] = input[i];
    }

    else
    {
        if (j != 0)
        {
            num[j] = '\0';

            sum += strtol(num, NULL, 10);

            j = 0;
        }
    }
}

int file = open(filename, O_WRONLY | O_CREAT | O_APPEND, 0644);

if (file == -1)
{
    const char msg[] = "error: failed to open requested file\n";

    write(STDERR_FILENO, msg, sizeof(msg));

    munmap(shm_ptr, 1024);

    sem_close(semaphore);

    exit(EXIT_FAILURE);
}

char sum_str[60];

snprintf(sum_str, sizeof(sum_str), "%ld -- sum\n", sum);

int32_t written = write(file, sum_str, strlen(sum_str));

if (written != strlen(sum_str))
{
    snprintf(sum_str, sizeof(sum_str) - 1, "error: failed to write to file\n");

    write(STDERR_FILENO, sum_str, sizeof(sum_str));

    exit(EXIT_FAILURE);
}

```

```
    }

    close(file);

    munmap(shm_ptr, 1024);

    sem_close(semaphore);

    return 0;
}
```

### **posix\_ipc-server.c**

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <fcntl.h>

#include <sys/mman.h>

#include <semaphore.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <unistd.h>

#include <stdint.h>

#define SHM_SIZE 1024

#define SEM_NAME "/my_semaphore"

#define SHM_NAME "/my_shared_memory"

static char CLIENT_PROGRAM_NAME[] = "posix_ipc-client";

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
```

```

    char msg[1024];

    int len = snprintf(msg, sizeof(msg), "usage: %s filename\n", argv[0]);

    write(STDERR_FILENO, msg, len);

    exit(EXIT_FAILURE);
}

int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);

if (shm_fd < 0)
{
    perror("error: failed to open shared memory");

    exit(EXIT_FAILURE);
}

if (ftruncate(shm_fd, SHM_SIZE) == -1)
{
    perror("error: failed to ftruncate");

    shm_unlink(SHM_NAME);

    exit(EXIT_FAILURE);
}

char *shm_ptr = mmap(0, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd,
0);

if (shm_ptr == MAP_FAILED)
{
    perror("error: failed to map shared memory");

    shm_unlink(SHM_NAME);

    exit(EXIT_FAILURE);
}

sem_t *semaphore = sem_open(SEM_NAME, O_CREAT, 0644, 0);

if (semaphore == SEM_FAILED)

```



```

{

    perror("error: failed to sem_open");

    munmap(shm_ptr, SHM_SIZE);

    shm_unlink(SHM_NAME);

    exit(EXIT_FAILURE);

}


pid_t pid = fork();

if (pid < 0)

{

    perror("error: failed to fork");

    munmap(shm_ptr, SHM_SIZE);

    shm_unlink(SHM_NAME);

    sem_close(semaphore);

    sem_unlink(SEM_NAME);

    exit(EXIT_FAILURE);

}


if (pid == 0)

{

    char shm_fd_str[10];

    sprintf(shm_fd_str, "%d", shm_fd);

    char path[1050];

    snprintf(path, sizeof(path), "%s/%s", ".", CLIENT_PROGRAM_NAME);

    char *const args[] = {CLIENT_PROGRAM_NAME, argv[1], NULL};

    execv(path, args);

    perror("error: failed to exec into new executable image");

```

```

munmap(shm_ptr, SHM_SIZE);

shm_unlink(SHM_NAME);

sem_close(semaphore);

sem_unlink(SEM_NAME);

exit(EXIT_FAILURE);

}

else

{

    printf("Введите числа через пробел: ");

    fgets(shm_ptr, SHM_SIZE, stdin);

    if (sem_post(semaphore) == -1)

    {

        perror("error: failed to sem_post");

    }

    wait(NULL);

    munmap(shm_ptr, SHM_SIZE);

    shm_unlink(SHM_NAME);

    sem_close(semaphore);

    sem_unlink(SEM_NAME);

}

return 0;

}

```

## **Протокол работы программы**

**Тестирование:**

```
ali_@LAPTOP-TG8SK2OI:~/OS_2/lab_3/src$ ./server test1.txt
```

Введите числа через пробел: 3 4 5

```
ali_@LAPTOP-TG8SK2OI:~/OS_2/lab_3/src$ cat test1.txt
```

```
12 -- sum
```

### Starce 1:

```
ali_@LAPTOP-TG8SK2OI:~/OS_2/lab_3/src$ strace ./server test1.txt
```

```
execve("./server", ["/server", "test1.txt"], 0x7ffda541ea8 /* 28 vars */) = 0
```

```
brk(NULL) = 0x7ffdc43b000
```

```
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffe34dbf40) = -1 EINVAL (Invalid argument)
```

```
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe2368d0000
```

```
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
```

```
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
```

```
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=17939, ...}, AT_EMPTY_PATH) = 0
```

```
mmap(NULL, 17939, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fe23688b000
```

```
close(3) = 0
```

```
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
```

```
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0"..., 832) = 832
```

```
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
```

```
pread64(3, "\4\0\0\0\0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0"..., 48, 848) = 48
```

```
pread64(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\0I\17\357\204\3$\f221\2039x\324\224\323\236S"..., 68, 896) = 68
```

```
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...}, AT_EMPTY_PATH) = 0
```

```
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
```

mmap(NULL, 2264656, PROT\_READ, MAP\_PRIVATE|MAP\_DENYWRITE, 3, 0) =  
0x7fe236660000

mprotect(0x7fe236688000, 2023424, PROT\_NONE) = 0

mmap(0x7fe236688000, 1658880, PROT\_READ|PROT\_EXEC,  
MAP\_PRIVATE|MAP\_FIXED|MAP\_DENYWRITE, 3, 0x28000) = 0x7fe236688000

mmap(0x7fe23681d000, 360448, PROT\_READ,  
MAP\_PRIVATE|MAP\_FIXED|MAP\_DENYWRITE, 3, 0x1bd000) = 0x7fe23681d000

mmap(0x7fe236876000, 24576, PROT\_READ|PROT\_WRITE,  
MAP\_PRIVATE|MAP\_FIXED|MAP\_DENYWRITE, 3, 0x215000) = 0x7fe236876000

mmap(0x7fe23687c000, 52816, PROT\_READ|PROT\_WRITE,  
MAP\_PRIVATE|MAP\_FIXED|MAP\_ANONYMOUS, -1, 0) = 0x7fe23687c000

close(3) = 0

mmap(NULL, 12288, PROT\_READ|PROT\_WRITE, MAP\_PRIVATE|MAP\_ANONYMOUS, -1,  
0) = 0x7fe236650000

arch\_prctl(ARCH\_SET\_FS, 0x7fe236650740) = 0

set\_tid\_address(0x7fe236650a10) = 8679

set\_robust\_list(0x7fe236650a20, 24) = 0

rseq(0x7fe2366510e0, 0x20, 0, 0x53053053) = -1 ENOSYS (Function not implemented)

mprotect(0x7fe236876000, 16384, PROT\_READ) = 0

mprotect(0x7fe2368d9000, 4096, PROT\_READ) = 0

mprotect(0x7fe2368c8000, 8192, PROT\_READ) = 0

prlimit64(0, RLIMIT\_STACK, NULL, {rlim\_cur=8192\*1024, rlim\_max=8192\*1024}) = 0

munmap(0x7fe23688b000, 17939) = 0

**openat(AT\_FDCWD, "/dev/shm/my\_shared\_memory",  
O\_RDWR|O\_CREAT|O\_NOFOLLOW|O\_CLOEXEC, 0666) = 3**

ftruncate(3, 1024) = 0

**mmap(NULL, 1024, PROT\_READ|PROT\_WRITE, MAP\_SHARED, 3, 0) = 0x7fe2368d5000**



`wait4(-1, NULL, 0, NULL) = 8685`

`--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=8685, si_uid=1000,  
si_status=0, si_etime=0, si_stime=0} ---`

`munmap(0x7fe2368d5000, 1024) = 0`

`unlink("/dev/shm/my_shared_memory") = 0`

`munmap(0x7fe2368d4000, 32) = 0`

`unlink("/dev/shm/sem.my_semaphore") = 0`

`exit_group(0) = ?`

`+++ exited with 0 +++`

## **Вывод**

Я приобрела навыки управления процессами в ОС и обеспечение обмена данных между процессами посредством shared memory. Научилась использовать shared memory и memory mapping. Отработала использование семафоров для синхронизации работы с общей памятью. Отладила программу на языке Си.